Python Data Structures - Complete Guide

A comprehensive guide to Python's built-in data structures including Strings, Lists, Tuples, Dictionaries, and Sets with complete practice solutions.

1️⃣ Strings in Python

📌 **Definition:** A string is an **immutable sequence of characters** in Python. Once created, individual characters cannot be modified.

Basic String Operations

```
Word1 = "Python" Word2 = " With Sonu Saini" # Length of string
print(len(Word1)) # Output: 6 # Concatenation print(Word1 + Word2) #
Output: Python With Sonu Saini # Accessing characters (indexing starts from
0) print(Word1[0]) # Output: P # Iterating through string for ch in Word2:
print(ch)
```

> **Important:** Strings are immutable. Operations like `Word[1] = 'P'` will raise an error.

String Slicing

✂️ **Syntax:** `str[starting_index : ending_index]`
- Starting index: default is `0`
- Ending index: **excluded** from result, default is `len(str)`
- Negative indexing: `-len(str)` to `-1` (from end)

```
print(Word1[2:5]) # Output: tho print(Word2[2:]) # From index 2 to end
print(Word2[:6]) # From start to index 5 print(Word2[-5:-2]) # Negative
indexing from end
```

String Indexing Diagram

```
Positive Index: 0 1 2 3 4 5 P y t h o n Negative Index: -6 -5 -4 -3 -2 -1
```

String Formatting

🎨 **Dynamic Strings:** Create strings with variable values using placeholders `{}`
Two Methods: `format()` method and **f-strings**

Method 1: Using format()

```
a = 5 b = 10 sum = a + b # Normal formatting print("sum of {} and {} is {}".format(a, b, sum)) print("language is {}".format("Python")) # Index-based formatting print("Sum is {2} of {0} and {1}".format(a, b, sum)) # Value-based formatting (reusability) print("values of vars {a} & {b} where a is {a}".format(a=5, b=10))
```

Method 2: F-Strings (Literal String Interpolation)

```
print(f"Literal string interpolation Sum of {a} and {b} is {a+b}")
```

> **Best Practice:** F-strings are more readable and faster than `format()`

2️⃣ Lists in Python

📌 **Definition:** A list is a **mutable sequence of values** that can store elements of different data types. Lists maintain order and allow indexing.

Creating and Accessing Lists

```
marks = [34, 53, 56, 33, 56, 54] print(marks) # Full list print(len(marks)) # Length: 6 print(marks[1]) # Access element: 53 # Lists are mutable marks[2] = 90 # ✓ This works! # Mixed data types detail = ["Sonu", 99, "Lalit", 98] print(type(list)) # <class 'list'>
```

List Slicing

```
print(marks[0:5]) # First 5 elements print(marks[5:len(marks)]) # From index 5 to end print(marks[-5:-1]) # Negative indexing
```

Essential List Methods

Method	Description	Example
`append(val)`	Add one element at the end	`numbers.append(6)`
`insert(idx, val)`	Insert element at specific index	`numbers.insert(2, 9)`
`sort()`	Arrange in ascending order	`numbers.sort()`
`sort(reverse=True)`	Arrange in descending order	`numbers.sort(reverse=True)`
`reverse()`	Reverse the current order	`numbers.reverse()`

Practical Example

```
numbers = [1, 2, 3, 4] numbers.append(6) print(numbers) # [1, 2, 3, 4, 6] numbers.insert(2, 9) print(numbers) # [1, 2, 9, 3, 4, 6] numbers.sort() print(numbers) # [1, 2, 3, 4, 6, 9] numbers.sort(reverse=True) print(numbers) # [9, 6, 4, 3, 2, 1] numbers.reverse() print(numbers) # [1, 2, 3, 4, 6, 9]
```

Using Loops with Lists

```
# Iterating through list for val in numbers: print(val) # Linear Search Example num = int(input("Enter the number to search: ")) idx = 0 for val in numbers: if val == num: print(f"Value {num} found at index -> {idx}") break idx += 1
```

3️⃣ Tuples in Python

📌 **Definition:** A tuple is an **immutable sequence of values** created using parentheses `()`. Once created, elements cannot be modified, added, or removed.

Creating and Accessing Tuples

```
tupple = (1, 2, 3, 54, 5, 6, "Sonu", "Lalit", 1.10) print(tupple) print(type(tupple)) # <class 'tuple'> print(len(tupple)) # Length: 9 print(tupple[6]) # Access: "Sonu" # tupple[4] = 9 # ❌ Error! Tuples are immutable
```

Single-Value Tuple Gotcha

⚠️ **Important:** Single-value tuples need a trailing comma, otherwise Python interprets them as that data type!

```
singletupple = (2) print(type(singletupple)) # <class 'int'> ❌
singletupple = ("Sonu") print(type(singletupple)) # <class 'str'> ❌
singletup = (1,) # Note the comma! print(type(singletup)) # <class 'tuple'>
✓
```

Tuple Slicing and Iteration

```
print(tupple[:5]) # First 5 elements print(tupple[:]) # All elements for
var in tupple: print(var)
```

Tuple Methods

Method	Description	Example
`index(val)`	Return index of first occurrence	`mytup.index(2)`
`count(val)`	Count total occurrences	`mytup.count(2)`

```
mytup = (2, 4, 2, 5, 6, "sonu", "Sonu") print(f"index of 2 is
{mytup.index(2)}") # 0 print(f"no. of times value 2 in tupple is
{mytup.count(2)}") # 2
```

List vs Tuple Comparison

```
┌──────────────┬──────────────┬──────────────┐ │ Feature │ List │ Tuple │
├──────────────┼──────────────┼──────────────┤ │ Syntax │ [ ] │ ( ) │ │
Mutable │ Yes ✓ │ No ✗ │ │ Speed │ Slower │ Faster │ │ Use Case │ Dynamic
│ Fixed Data │ └──────────────┴──────────────┴──────────────┘
```

4️⃣ Dictionaries in Python

📌 **Definition:** A dictionary is a collection of **key-value pairs**. Keys must be **unique** and **immutable**. Dictionaries are **mutable** and **unordered**.

Creating and Using Dictionaries

```
info = { "name": "Sonu", "Cgpa": 9.8, "Subjects": ["maths", "Science",
"English"], 3.14: "PI" # Keys can be numbers too } print(info)
print(info["name"]) # Access value: "Sonu" # Modify value (dictionaries are
mutable) info["Cgpa"] = 9.9 print(info)
```

Dictionary Methods

Method	Description	Returns
`keys()`	Get all keys	dict_keys object
`values()`	Get all values	dict_values object
`items()`	Get (key, value) pairs	dict_items object
`get(key)`	Get value by key	Value or None
`update(dict)`	Add new items	None

Working with Dictionary Methods

```
dict_key = list(info.keys()) print(dict_key) # ['name', 'Cgpa', 'Subjects', 3.14] print(type(dict_key)) # <class 'list'> dict_val = info.values() print(dict_val) # dict_values(['Sonu', 9.9, ...]) print(type(dict_val)) # <class 'dict_values'> print(info.items()) # All key-value pairs
```

get() vs [ ] Notation

💡 `get()` **advantage:** Returns `None` if key doesn't exist (no error)
`[]` **notation:** Raises `KeyError` if key doesn't exist

```
print(info.get("Subjects")) # Works: returns list print(info.get("no_key")) # Returns: None (no error) # Adding new items info.update({ "city": "Delhi" }) print(info)
```

5️⃣ Sets in Python

📌 **Definition:** A set is a collection of **unique elements**. Set elements must be **immutable**, but the set itself is **mutable**. Sets are **unordered** (no indexing).

Creating and Using Sets

```
myset = {22, 2, 3, 34, 4, 2, 2} print(myset) # {2, 3, 4, 22, 34} - duplicates removed! print(type(myset)) # <class 'set'> print(len(myset)) # 5 myset.add(44) print(myset)
```

Creating Empty Sets

⚠️ **Warning:** `{}` creates an empty **dictionary**, not a set!
Use `set()` to create an empty set.

```
empty_di = {} print(type(empty_di)) # <class 'dict'> emptyset = set() print(type(emptyset)) # <class 'set'>
```

Set Methods

Method	Description	Example
`add(val)`	Add an element	`myset.add(67)`
`remove(val)`	Remove an element	`myset.remove(67)`
`clear()`	Empty the set	`myset.clear()`
`pop()`	Remove random value	`s.pop()`
`union(set2)`	Combine two sets	`a.union(b)`
`intersection(set2)`	Common elements	`a.intersection(b)`

Practical Examples

```
myset.add(67) print(myset) # {2, 3, 4, 22, 34, 44, 67} myset.remove(67)
print(myset) # {2, 3, 4, 22, 34, 44} myset.clear() print(myset) # set() s =
{34, 45, 76, 4, 34, 3, 23, 4} print(s) s.pop() # Removes random element
print(s)
```

Set Operations

```
a = {2, 2, 3, 3445, 56, 6, 7} b = {3, 2, 4, 45, 6, 7, 889, 8} # Union: All
unique elements from both sets print(a.union(b)) # {2, 3, 4, 6, 7, 8, 45,
56, 889, 3445} # Intersection: Common elements only
print(a.intersection(b)) # {2, 3, 6, 7}
```

Set Operations Diagram

```
Set A: {2, 3, 56, 6, 7, 3445} Set B: {2, 3, 4, 6, 7, 8, 45, 889} Union (A ∪ B): {2, 3, 4, 6, 7, 8, 45, 56, 889, 3445} Intersection (A ∩ B): {2, 3, 6, 7}
```

🎯 Practice Problems - Complete Solutions

📝 **Problem:** Given a list of tuples with student-course info:
1. List all unique courses
2. List students enrolled in English
3. Create dictionary mapping students to sets of courses

Data Setup

```
mylist = [ ("Sonu", "Maths"), ("Lalit", "Science"), ("Sonu", "Science"), ("Nikhil", "Maths"), ("Lalit", "Maths"), ("Sonu", "English"), ("Nikhil", "English") ]
```

Solution 1: List All Unique Courses

```
unique_courses = set() for tup in mylist: unique_courses.add(tup[1])
print(unique_courses) # Output: {'Maths', 'Science', 'English'}
```

💡 **Key Concept:** Using a **set** automatically handles uniqueness for us!

Solution 2: List Students Enrolled in English

✨ **Tip:** When working with iterables (objects you can loop through), you can unpack variables directly in the loop: `for name, course in mylist`

```
# Using tuple unpacking with iterables for name, course in mylist: if course == "English": print(name, course) # Output: # Sonu English # Nikhil English
```

Solution 3: Create Dictionary (Student → Set of Courses)

```
dict = {} for name, course in mylist: if dict.get(name) == None: # Student not in dictionary yet, create new entry dict.update({name: set()}) dict[name].add(course) else: # Student already exists, add course to their set dict[name].add(course) print(dict)
```

Output:

```
{ 'Sonu': {'Maths', 'Science', 'English'}, 'Lalit': {'Science', 'Maths'}, 'Nikhil': {'Maths', 'English'} }
```

How It Works:

Step-by-step breakdown:

1️⃣ **Initialize empty dictionary:** `dict = {}`

2️⃣ **Loop through each tuple:** Extract name and course using unpacking

3️⃣ **Check if student exists:**

- If `dict.get(name)` returns `None` → student not in dictionary
- Create new entry with an empty set: `{name: set()}`
- Add the current course to the set

4️⃣ **If student already exists:**

- Simply add the course to their existing set
- Sets automatically prevent duplicates!

Alternative Approach (More Pythonic):

```
from collections import defaultdict student_courses = defaultdict(set) for
name, course in mylist: student_courses[name].add(course)
print(dict(student_courses))
```

🚀 **Advanced Tip:** `defaultdict(set)` automatically creates an empty set for new keys, eliminating the need for the None check!